

CONCEPTOS DE BASES DE DATOS

CLASE 7



Dispersión

- Mediante el uso de **índices** y de árboles balanceados **B**, **B*** y **B+**, es posible acceder eficientemente a cualquier registro de un archivo, generalmente en una **pequeña cantidad de accesos** a almacenamiento secundario.
- En muchos casos, requerir 3 o 4 accesos a disco representa un **rendimiento muy satisfactorio**, especialmente cuando además se necesita mantener los datos ordenados o realizar búsquedas por rango.

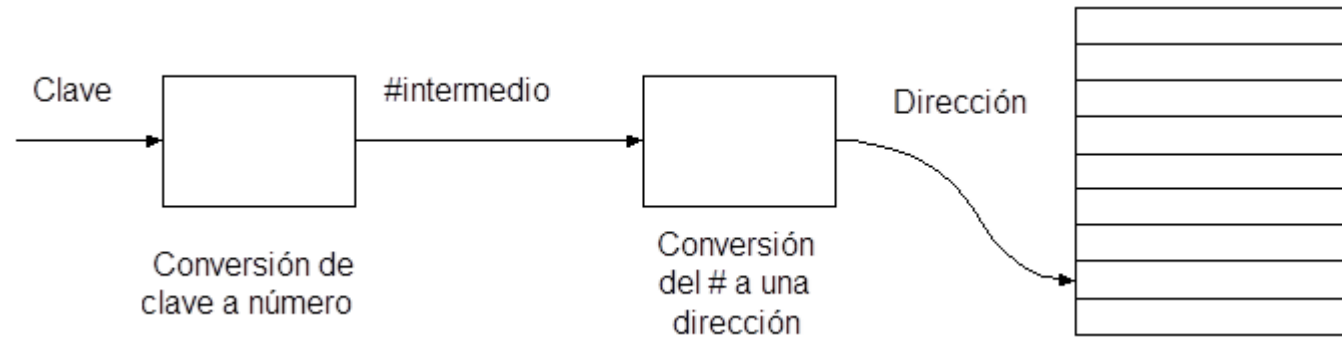
Dispersión

- Pero existen situaciones donde se prioriza obtener un **registro específico en el menor tiempo posible**, idealmente con una única lectura.
- Esto suele ocurrir en sistemas con:
 - Gran volumen de consultas puntuales.
 - Respuestas en tiempo real.
 - Alta concurrencia de usuarios.
 - Transacciones masivas de lectura/escritura.
- Para estos casos, se necesita un mecanismo de acceso a registros con una **única lectura** -> **DISPERSIÓN**

Dispersión

- La dispersión es una técnica de almacenamiento y recuperación de datos, que sirve para generar una **dirección base única** para una clave dada.
- Convierte la clave del registro en un **número aleatorio**, el que sirve después para determinar donde se almacena el registro.
- Utiliza lo que se denomina **función de hash** para mapear los registros en direcciones de almacenamiento.

Dispersión



- Para determinar la dirección de un registro:
 - La clave se convierte en un **número aleatorio**.
 - El número se convierte en una **dirección de memoria**.
 - El registro **se debe guardar en esa dirección**.
 - Si la dirección está completa: **saturación (overflow)** → **tratamiento especial**.

Dispersión

- Beneficios

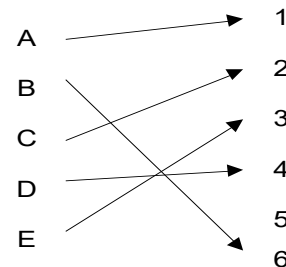
- No requiere almacenamiento adicional (índices).
- Facilita inserción y eliminación rápida de registros.
- Encuentra registros con muy pocos accesos al disco en promedio (generalmente menos de 2).

- Costos

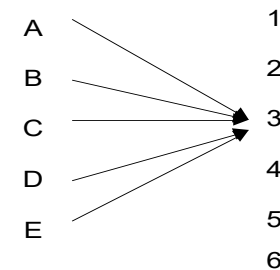
- No es posible el uso de registros de longitud variable.
- No existe el orden físico de datos.
- No permite claves duplicadas.

Dispersión

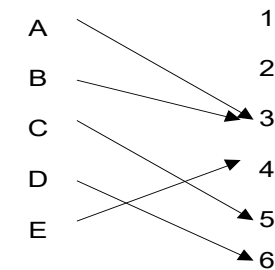
- Algoritmos de dispersión
 - **Uniforme**: reparte los registros en forma uniforme en el espacio de direcciones disponible → **difícil de lograr**.
 - **Aleatoria**: las claves son independientes, no influyen una sobre la otra. Cualquier dirección tiene la misma probabilidad de ser elegida para una clave.



uniforme



peor



aceptable

Dispersión

- Tipos de dispersión
 - Con espacio de direccionamiento **estático**.
 - El espacio disponible para dispersar los registros esta **prefijado**.
 - Con espacio de direccionamiento **dinámico**.
 - El espacio disponible para dispersar los registros **aumenta o disminuye según las necesidades** de espacio que en cada momento tiene el archivo.

Dispersión

Estático

- Parámetros que afectan la **eficiencia**
 - Función de dispersión.
 - Tamaño de los compartimentos (espacio de almacenamiento).
 - Densidad de empaquetamiento.
 - Método de tratamiento de saturación.

- **Función de dispersión**
 - **Caja negra:** a partir de una clave se obtiene la dirección donde debe estar el registro.
 - **Diferencias con índices:**
 - No hay relación aparente entre la clave y la dirección.
 - Dos claves distintas pueden transformarse en una misma dirección → **colisión**.
 - **Resultado:** retorna un **valor aleatorio**, que luego **debe ser mapeado** al rango de valores disponibles.

- Función de dispersión → ejemplo sencillo
 - Para una clave de tipo string, en un archivo que tiene **1000 direcciones** asignadas (0..999). Para comprender la idea general, se considera una **función muy sencilla**.
 - La dirección de almacenamiento se obtiene de la sig. forma:
 - Se toma el **código ASCII de los dos primeros caracteres en mayúscula** de la clave.
 - Se **multiplican** ambos valores.
 - Se trunca el resultado obtenido: se consideran los **últimos 3 dígitos**.
 - La dirección de una clave con valor **LOMAS**:
 - $L = 76, O = 79 \rightarrow 76 \times 79 = 6004$
 - Dirección resultante: **004**.

Dispersión

Estático

- Función de dispersión → ejemplo sencillo
 - Pero, la dirección de una clave con valor **OLIVOS**:
 - $O = 79, L = 76 \rightarrow 79 \times 76 = 6004$
 - Dirección resultante: **004**.
 - La misma dirección obtendrán claves como **LOBERÍA, LOMAS, OLAVARRÍA, OLMOS**, entre otras.
 - La misma dirección obtendrán claves cuyo múltiplo de sus 2 caracteres iniciales de como resultado **5004, 6004, 7004**.

- **Función de dispersión** → tipos de funciones
 - **Centros cuadrados:** la clave se multiplica por sí misma y se toma los dígitos centrales, posteriormente se ajusta al espacio disponible.
 - **División (módulo):** la clave se divide por un nro. aproximadamente igual a la cantidad de direcciones (número primo → tiende a distribuir residuos en forma más eficiente). Luego se obtiene el residuo.
 - **Desplazamiento:** se realiza un desplazamiento del número, se particiona y se suman las partes resultantes. Luego se ajusta al espacio disponible.
 - **Plegado:** se realiza el plegado del número, se suman las partes resultantes y adaptan al espacio de direcciones.
 - **Transformación de base:** la base del número se modifica y luego se toma el resultado del módulo.

Dispersión

Estático

- Función de dispersión → Centros cuadrados
 - N= 7000 (direcciones)
 - Clave: 6 dígitos

Clave = 172148

Clave ² = 029634933904 → 3493 * 0.7 = 2445

Para llevar al espacio asignado al archivo multiplico por 0.7 (ya que haciendo el cálculo con la mayor clave posible: 7000/9999 = 0.7).

Dispersión

Estático

- Función de dispersión → División (módulo)
 - N= 7000 (direcciones)
 - Clave: 6 dígitos

Clave = 172148

Clave mod 6997 = 4220 → El resultado siempre estará entre 0 y 6996.

Dispersión

Estático

- **Función de dispersión** → **Desplazamiento**
 - **N= 7000 (direcciones)**
 - **Clave: 8 dígitos**

Clave = 20175973

Clave desplazada= 17207359 = 1720 + 7359 → 9079 * 0.35 = 3177

Para llevar al espacio asignado al archivo multiplico por 0.35 (ya que haciendo el cálculo con la mayor clave posible: $7000/19998 = 0.35$).

Dispersión

Estático

- Función de dispersión → Plegado
 - N= 999 (direcciones)
 - Clave: 8 dígitos

Clave = 17207359

Clave plegada en 3 partes = 017 207 359 = 953 + 207 + 710 → 1870 * 0.33 = 617

Para llevar al espacio asignado al archivo multiplico por 0.33 (ya que haciendo el cálculo con la mayor clave posible: $999/2988 = 0.33$).

Dispersión

Estático

- Función de dispersión → Transformación de base
 - N= 100 (direcciones)
 - Clave: 3 dígitos

Clave = 453

Clave en base 11 = 382 → $382 * 0.12 = 45$

Para llevar al espacio asignado al archivo multiplico por 0.12 (ya que haciendo el cálculo con la mayor clave posible: $100/829 = 0.12$).

- Función de dispersión
 - ¿Cuál elegir?
 - Tomar algunas claves del archivo y **simular el comportamiento** con algunos métodos, y luego elegir el que **mejor** se comporte.
 - En general:
 - **División** mejor.
 - **Plegado**, para claves muy largas.

Dispersión

Estático

- Colisiones

- La función de dispersión asocia cada clave con una dirección de almacenamiento. Puede ocurrir que **la dirección calculada ya se encuentre ocupada** por otro registro.
- Cuando dos o más claves distintas son enviadas a la misma posición, se produce una **colisión**.
- Se denomina **sinónimos** a aquellas claves diferentes que, al aplicar la función hash, generan la misma dirección.

- Colisiones
 - ¿Se pueden evitar por completo? En la práctica, **no existe un algoritmo de dispersión perfecto** que garantice ausencia total de colisiones para cualquier conjunto de claves.
 - Esto se debe a que normalmente:
 - La **cantidad de claves** posibles es **muy grande**.
 - La **cantidad de direcciones** disponibles es **limitada**.
 - El **objetivo principal** es entonces **reducir la cantidad de colisiones** y lograr una distribución uniforme de las claves.

Dispersión

Estático

- Colisiones → disminución
 - **Esparcir registros:** buscar métodos que distribuyan los registros de la forma **más aleatoria posible** entre las direcciones disponibles.
 - Producto de códigos ASCII → **no es tan eficiente**.
 - Las otras funciones vistas → elegir la **mejor** para cada caso particular.
 - **Usar memoria adicional:** distribuir pocos registros en muchas direcciones (ej: 75 registros en 1000 direcciones).
 - **Disminuye** las colisiones.
 - **Desperdicia** espacio.

Dispersión

Estático

- Colisiones → disminución
 - Colocar más de un registro por dirección
 - Las direcciones capaces de almacenar uno o varios registros se denominan cubetas o compartimentos.
 - Si cada dirección admite varios registros:
 - Se disminuye la necesidad de desbordes inmediatos.
 - Mejora el rendimiento general del archivo disperso.
 - Ej: archivo con direcciones de 512 bytes, donde cada registro ocupa 80 bytes → hasta 6 registros por cada dirección.

- Tamaño de los compartimentos
 - Entonces, ¿conviene usar cubetas **muy grandes**?
 - Aumentar la capacidad de cada compartimento **permite absorber más colisiones**, podría pensarse que siempre es conveniente para almacenar muchos registros sin generar desbordes.
 - Sin embargo, en la práctica esto también introduce **costos y limitaciones**.

- Tamaño de los compartimentos
 - **Ventajas** del uso de cubetas grandes:
 - Aumenta la cantidad de registros que pueden almacenarse en una cubeta.
 - Disminuye la probabilidad de saturación.
 - **Desventajas** del uso de cubetas grandes:
 - Búsqueda interna más lenta.
 - Hay mas espacios vacíos, por lo que aumenta la fragmentación.

- Tamaño de los compartimentos
 - Además, el tamaño máximo del compartimento no depende solamente del **diseño lógico** del archivo.
 - También está **condicionado por aspectos físicos y del sistema operativo**, como:
 - Buffers disponibles en RAM.
 - Tamaño de página administrado por el SO.
 - Capacidad de transferencia en operaciones de E/S.
 - Rendimiento del hardware de almacenamiento.

- Tamaño de los compartimentos
 - Entonces: ¿cuál es el tamaño que se debe utilizar?
 - Normalmente se elige un tamaño de cubeta **compatible con los buffers del sistema.**
 - De esta forma, cada acceso permite transferir una cubeta completa en una sola operación de entrada/salida, mejorando notablemente la eficiencia.

- **Densidad de empaquetamiento**
 - En archivos dispersos, no solo importa la cantidad total de direcciones disponibles, sino también **qué tan aprovechado se encuentra el espacio asignado**.
 - Para medir ese nivel de utilización se emplea el concepto de densidad de empaquetamiento (**DE**).
 - La **DE** representa la proporción del espacio asignado al archivo que **realmente está ocupado por registros**.
 - No es relevante el tamaño total del archivo, sino la **relación entre capacidad disponible y ocupación real**.

- Densidad de empaquetamiento
 - $DE = \frac{\text{número de registros almacenados en el archivo}}{\text{capacidad total de las cubetas}}$
 - Entonces:
 - Si $DE = 1 \rightarrow$ el archivo está completamente lleno.
 - Si $DE = 0.7 \rightarrow$ se utiliza el 70% de la capacidad disponible.
 - Si $DE = 0.3 \rightarrow$ gran parte del espacio permanece libre.
 - Si se cuenta con una **DE menor**:
 - Menor saturación.
 - Mayor desperdicio de espacio.

Dispersión

Estimación overflow

- El **riesgo de saturación de las cubetas** puede analizarse a partir de las **características físicas del archivo** y de la **cantidad de registros almacenados**.
- Conociendo la capacidad disponible y el volumen de datos, es posible realizar una estimación del nivel de ocupación esperado.
- Se puede estimar la cantidad de registros que deberán ser almacenados en una dirección diferente a su dirección base.

Dispersión

Estimación overflow

- Sea:
 - N = número de cubetas del archivo.
 - C = capacidad de cada cubeta.
 - K = cantidad total de registros almacenados.

- Entonces:

$$DE = \frac{K}{C \times N}$$

- ¿Cuál es la probabilidad de que una cubeta en particular reciba una cantidad determinada de registros?

Dispersión

Estimación overflow

- Estimación de la cantidad de claves que recibe una cubeta
 - Dadas:
 - A = no se utiliza una cubeta en particular.
 - B = se utiliza una cubeta en particular.
 - Para una clave:
 $P(B) = 1/N$ $P(A) = 1 - P(B) = 1 - 1/N$
 - Para dos claves:
 $P(BB) = P(B) * P(B) = (1/N)^2$
 $P(BA) = P(B) * P(A) = (1/N) * (1 - 1/N)$

Nota: se asume que cada clave es independiente y que la función de hash es uniforme.

Dispersión

Estimación overflow

- Estimación de la cantidad de claves que recibe una cubeta
 - $P(\text{secuencia}) = P(B)^{\#B} * P(A)^{\#A}$
 - Para una secuencia determinada de K claves, la probabilidad de que I caigan en una cubeta es:

$$(1/N)^I * (1 - 1/N)^{K-I}$$

- La cantidad de formas de **combinar** esa secuencia es (K tomadas de a I combinaciones):

$$\frac{K!}{I! * (K - I)!}$$

Dispersión

Estimación overflow

- Estimación de la cantidad de claves que recibe una cubeta
 - La probabilidad de que una cubeta reciba **I** elementos de los **K** disponibles es entonces:

$$P(I) = \underbrace{\frac{K!}{I! * (K-I)!}}_{\text{COMBINATORIO } \binom{K}{I}} * \underbrace{\left(\frac{1}{N}\right)^I * \left(1 - \frac{1}{N}\right)^{K-I}}_{\substack{P(B) * P(A) \\ I \text{ VECES } \quad K-I \text{ VECES}}}$$

- Existe una **aproximación mas sencilla** que esta fórmula, obtenida mediante el uso de una distribución de Poisson:

$$P(I) = \frac{(K / N)^I * e^{-(K / N)}}{I!}$$

Dispersión

Estimación overflow

- En general, si hay **N** direcciones → el nro esperado de direcciones con **I** registros asignados es igual a **N***P(**I**).
- Se analizarán archivos con **diferentes DE**, estimando la saturación en cada caso.
- Ejemplo 1: **N = 10000** **K = 10000** **DE = 1**
 - La proporción de direcciones con 0 registros asignados es:

$$P(0) = \frac{(10.000 / 10.000)^0 * e^{-(10.000/10.000)}}{0!} = 0.3679$$

Dispersión

Estimación overflow

- Ejemplo 1: **N** = 10000 **K** = 10000 **DE** = 1
 - Nº de direcciones **sin registros** asignados (**I** = 0):
0: $10000 * P(0) = 10000 * 0.3679 = 3679$
 - Nº de direcciones **con uno, dos y tres registros**:
1: $10000 * P(1) = 10000 * 0.3679 = 3679$
2: $10000 * P(2) = 10000 * 0.1839 = 1839$
3: $10000 * P(3) = 10000 * 0.0613 = 613$

Dispersión

Estimación overflow

- Ejemplo 1: $N = 10000$ $K = 10000$ $DE = 1$
 - La cantidad de registros en colisión será entonces:
 - 3679 direcciones tienen 0 registros asignados $\rightarrow 0$
 - 3679 direcciones tienen 1 registro asignado $\rightarrow 0$
 - 1839 direcciones tienen 2 registros asignados $\rightarrow 1839$
 - 613 direcciones tienen 3 registros asignados $\rightarrow (613 * 2)$
 - Saturación estimada hasta el momento: $1839 + (613 * 2) = 3065$.
 - Aún no se finalizó la estimación y **ya es mayor a un 30%**.
 - Más del 30% de los registros serán almacenados en algún lugar que **no es su dirección base**.

Dispersión

Estimación overflow

- Ejemplo 2: **N** = 1000 **K** = 500 **DE** = 0.5
 - Nº de direcciones **sin registros** asignados (**I** = 0):
0: $1000 * P(0) = 1000 * 0.607 = 607$
 - Nº de direcciones **con un sólo registro**:
1: $1000 * P(1) = 1000 * 0.303 = 303$
 - Nº de direcciones **con más de un registro**:
2: $1000 * P(2) = 1000 * 0.0758 = 75$
3: $1000 * P(3) = 1000 * 0.0126 = 12$
4: $1000 * P(4) = 1000 * 0.0016 = 1$
5: $1000 * P(5) = 1000 * 0.0002 = 0$

Dispersión

Estimación overflow

- Ejemplo 2: **N** = 1000 **K** = 500 **DE** = 0.5
 - ¿Cuántos registros en saturación pueden esperarse?
→ **N** * (1* P(2) + 2* P(3) + 3 * P(4) + 4 * P(5)) =
= 1000 * (1* 0.0758 + 2* 0.0126 + 3*0.0016 + 4*0.0002) =
= **107**
 - Dado que se cuenta con 500 registros en total, la saturación estimada hasta el momento es: **107/500** = **0.214 = 21.4 %**.
 - Más del **21%** de los registros serán almacenados en algún lugar que **no es su dirección base**.

Dispersión

Estimación overflow

- Valores de **saturación** para diferentes DE:

DE

Saturación

0.10

4.8 %

0.30

13.6 %

0.50

21.4 % → Ejemplo 2

0.70

28.1 %

0.80

31.2 %

0.90

34.1 %

1.00

36.8 % → Ejemplo 1

Dispersión

Estimación overflow

- En los ejemplos se utilizó compartimentos con capacidad para un **único registro**.
 - Solo en este caso, los términos de colisión y saturación son **equivalentes**.
- Los números bajos de saturación se presentan cuando se dispone de una baja DE.
 - **Disminuye la cantidad de colisiones.**
 - **Deja muchas direcciones (compartimentos) libres.**
- La solución a este bajo nivel de eficiencia implica el uso de compartimentos con capacidad para **más de un registro**.

Dispersión

Estimación overflow

- Si se consideran compartimentos con capacidad para **más de un registro**:
 - Dependiendo de su **capacidad** y del **lugar libre** que tenga un compartimento en un momento dado, **dos o más sinónimos se podrán almacenar en la misma dirección**.
 - Entonces ahora, al producirse una colisión:
 - Si el compartimento **tiene lugar suficiente** el registro se almacena en dicho lugar, por lo que **no hay saturación**.
 - Si el compartimento **está completo** (no tiene más lugar) entonces si **se produce saturación**.

Dispersión

Estimación overflow

K: Nº registros **B**: Tamaño de compartimento

N: Nº de direcciones de memoria disponibles

$$DE = K / B * N$$

	Archivo 1	Archivo 2
Nº registros (K)	750	750
Nº direcciones (N)	1000	500
Tamaño Comp. (B)	1	2
DE	0.75	0.75
Proporción regs/dirs	0.75	1.5

Dispersión

Estimación overflow

- Con 750 registros, 1000 direcciones de memoria disponibles y compartimentos de tamaño 1, el N° esperado de registros en saturación es:

$$\begin{aligned} &\rightarrow N * (1* P(2) + 2* P(3) + 3 * P(4) + 4 * P(5)) = \\ &= 1000 * (1*0.1328 + 2*0.0332 + 3*0.0062 + 4*0.0009 + \\ &5*0.0001) \\ &= 222 \end{aligned}$$

- Saturación = $222 / 750 = 29.6 \%$

Dispersión

Estimación overflow

- Con 750 registros, 500 direcciones de memoria disponibles y compartimentos de tamaño 2, el N° esperado de registros en saturación es:

$$\begin{aligned} &\rightarrow N * (1*P(3) + 2*P(4) + 3*P(5) + 4*P(6) + 5*P(7)) = \\ &= 500 * (1*0.1255 + 2*0.0471 + 3*0.0141 + 4*0.0035 + \\ &5*0.0008) = 140 \\ &= \mathbf{140} \end{aligned}$$

- Saturación = $140 / 750 = 18.7\%$ → mejora

Dispersión

Estimación overflow

- Probabilidad de saturación

	TAMAÑO COMPARTIMENTO				
DE	1	2	5	10	100
0.10	4.8	0.6	0.0	0.0	0.0
0.20	9.4	2.2	0.1	0.0	0.0
0.30	13.6	4.5	0.4	0.0	0.0
0.40	17.6	7.3	1.1	0.1	0.0
0.50	21.4	10.4	2.5	0.4	0.0
0.60	24.8	13.7	4.5	1.3	0.0
0.70	28.1	17.0	7.1	2.9	0.0
0.75	29.6	18.7	8.6	4.0	0.0
0.80	31.2	20.4	10.3	5.3	0.1
0.90	34.1	23.8	13.8	8.9	0.8
1.00	36.8	27.1	17.6	12.5	4.0